AgriFi Trade Network

AI Agent Build Specification

Purpose
A single, authoritative handoff document instructing an AI coding agent how to plan, structure, implement, test, and document the first release of AgriFi Trade Network.

Project decision	Agreed direction
Initial market	Nigeria first; design for later West African expansion
Backend	Python, Django, Django REST Framework
Primary database	PostgreSQL
Farmer mobile app	.NET MAUI + local SQLite for offline-first field workflows
Buyer / operations dashboard	Angular + Tailwind CSS
Async processing	Redis + Celery when needed; do not add complexity before a concrete use case
First-release payments	Simulated payment and escrow-style order workflow only
Delivery model	Solo developer; small, reviewable milestones

Document status: Implementation brief and acceptance contract. This is a design specification, not proof that the system has been implemented or production-tested.

Version 1.0 · 16 October 2026

Contents

1. Mission and product boundaries
2. Product roles and release scope
3. Technology stack and architecture
4. Repository and application structure
5. Domain model and database design
6. Business invariants and state machines
7. API contract and endpoint plan
8. Offline-first mobile synchronisation
9. Simulated payments and financial ledger
10. Security, privacy, reliability and operations
11. Implementation roadmap and agent workflow
12. Testing strategy and acceptance criteria
13. Explicit non-goals and future expansion
14. Instructions to the AI coding agent

How to use this document

Give this PDF to the coding agent as the primary project brief. The agent should inspect the existing repository first, report what is already present, reconcile differences against this specification, and then implement the next smallest complete milestone. Do not ask the agent to generate the whole system in one unreviewable pass.

1. Mission and product boundaries

AgriFi Trade Network is a business-to-business agricultural marketplace intended to help farmers and bulk buyers discover supply, record harvest availability, place traceable orders, coordinate fulfilment, and understand the status of payment-related workflows. The first market is Nigeria, but country-specific rules, currency, units, locations, and payment providers should be configurable rather than hard-coded throughout the codebase.

Product principles

- Trust before breadth: make inventory, order states, and financial records correct before adding advanced features.
- Server is authoritative: the backend is the source of truth for shared inventory, orders, payments, permissions, and posted ledger entries.
- Offline-aware, not offline-authoritative: the mobile app may capture harvest data without connectivity, but cannot promise stock to buyers until the server accepts the update.
- Modular monolith first: one Django deployment with clearly separated apps and service boundaries; no microservices in the first release.
- Traceability: important state changes and financial events must be auditable.
- Incremental delivery: every milestone must run, have tests, and be documented before the next begins.

First-release outcome

A user can register and be assigned an appropriate role; farmers can maintain crop listings and harvest batches; buyers can browse listings and place orders; inventory can be reserved without overselling; an order can progress through defined fulfilment states; payment can be simulated; and operators can inspect orders, stock, and payment records. The farmer mobile client can record data locally and synchronise it safely when connectivity returns.

Important legal and product boundary

The initial payment flow is simulated and escrow-style only. Do not accept, hold, transfer, or imply custody of real customer funds. Do not market the system as legally providing escrow. Any real payment gateway, regulated escrow arrangement, lending product, or cross-border settlement requires separate provider, legal, compliance, security, and product review.

2. Product roles and release scope

Role	Primary abilities	Restrictions
Farmer	Manage own profile, harvest batches and listings; view own orders and sync status	Cannot modify another farmer's records or manually post ledger entries
Buyer	Browse active listings; create and view own orders; use simulated payment flow	Cannot alter server-calculated prices, another buyer's orders, or inventory
Operations administrator	Review listings/users, manage allowed operational states, investigate disputes	Privileged actions must be permission-checked and audited
Finance operator	Inspect simulated payments, refunds and ledger reports; run approved simulation actions	Cannot edit or delete posted journal entries; no real-money transfer
System/service account	Run sync, expiry and background tasks where configured	Least privilege; credentials must not be exposed to clients

Release priorities

Priority	Scope
P0 — MVP	Authentication and roles; crop catalogue; farmer profiles; harvest batches; listings; inventory movements/reservations; single-seller orders; order status history; simulated payments; basic double-entry ledger; audit events; idempotent sync endpoint; basic web/admin workflows; tests and local setup.
P1 — After MVP	Notifications; improved analytics; operational reports; reservation-expiry worker; richer dispute handling; additional monitoring and deployment automation.
P2 — Future	SMS/USSD options; logistics integrations; credit assessment; real payment provider integration; real-money escrow arrangement; lending; multi-country/cross-border flows.

Keep the first checkout model simple

For MVP, each order belongs to one buyer and one seller. Each order item points to one listing. Do not implement multi-seller checkout initially. If required later, introduce a parent checkout/marketplace transaction with separate seller orders, rather than stretching a single seller order to represent multiple sellers.

3. Technology stack and architecture

Layer	Technology / decision
Backend API	Python; Django; Django REST Framework
Database	PostgreSQL as authoritative server database
Farmer mobile client	.NET MAUI; SQLite for local drafts and queued sync operations
Buyer / operations web client	Angular; Tailwind CSS
Background tasks	Celery + Redis when scheduled/async work is needed; initially keep deployment minimal
API format	Versioned REST API under /api/v1/; JSON; consistent error format
Authentication	Django-supported secure authentication approach; token strategy chosen and documented before client integration
Local development	Docker Compose recommended for PostgreSQL and optional Redis; environment-based settings
Quality	pytest or Django test runner, DRF API tests, formatting/linting, CI checks
Deployment	Documented repeatable deployment; secrets from environment/secret manager, not committed

Architecture pattern

Use a modular monolith. Django apps separate domain areas, but share one codebase and PostgreSQL database. Keep business rules in service functions/classes rather than embedding complicated workflows in views, serializers, or model signals. Views should authenticate, validate request shape, call services, and serialize results.

Suggested high-level flow

Angular / .NET MAUI clients → versioned DRF API → domain services → PostgreSQL. Optional Celery workers handle reservation expiry, notifications, and reports. Payment-provider logic must sit behind a payment adapter interface, even though the MVP uses only a simulated provider.

Dependencies policy

Prefer established, maintained packages and pin compatible versions. The agent must explain the purpose of each new dependency and avoid adding a library merely because it is fashionable. Do not introduce microservices, Kubernetes, event streaming, or complex cloud infrastructure for the MVP without a demonstrated need.

4. Repository and application structure

Use a clear, conventional layout. The agent should adapt this to an existing repository rather than overwrite files blindly.

```
agrifi-trade-network/ backend/ manage.py config/ settings/ urls.py asgi.py wsgi.py agrifi/ accounts/
catalog/ inventory/ orders/ payments/ ledger/ sync/ audit/ notifications/ # add when required
analytics/ # add when required tests/ requirements/ or pyproject.toml Dockerfile mobile/
AgriFi.Mobile/ # .NET MAUI app (phase after API foundations) web/ agrifi-dashboard/ # Angular +
Tailwind (phase after API foundations) docs/ product-requirements.md system-design.md
database-design.md api-contract.md decisions/ docker-compose.yml .env.example .gitignore README.md
```

Django app responsibilities

App	Owns
accounts	Custom User model, roles, farmer/buyer profiles, account permissions
catalog	Crop categories, listings, listing lifecycle and searchable catalogue
inventory	Harvest batches, stock movements, reservations, availability calculations
orders	Orders, order items, status transitions, status history, disputes
payments	Simulated payment attempts, payment status, refunds and settlement records
ledger	Ledger accounts, journal entries/lines, posting and reversal services
sync	Idempotent sync operations, sync receipts, conflict/error responses
audit	Append-only business audit events and audit helpers

Model and service conventions

- Use a custom Django user model before the first migration.
- Use UUID primary keys for business entities unless there is a documented reason not to.
- Use timezone-aware timestamps and Django's timezone utilities.
- Use DecimalField for quantities where fractional units are allowed; define precision, scale, and unit explicitly.
- Use integer minor units (for example, kobo) in BigIntegerField for monetary amounts; never use binary floating-point for money.
- Use TextChoices or equivalent enumerations for statuses, and database constraints for important row-level invariants.
- Do not use model signals as the only enforcement mechanism for financial or inventory workflows.
- Keep migrations small and committed; do not rewrite applied migrations casually.

5. Domain model and database design

Use PostgreSQL as the source of truth. The mobile SQLite database is a local cache/outbox and must not be treated as shared inventory authority.

Entity	Purpose / important fields
User	UUID; normalized unique email; password hash via Django auth; active flag; joined/updated timestamps
UserRole	User FK + role; unique pair; enforce role checks on the server
FarmerProfile	One-to-one user; display name; region/location; verification status
BuyerProfile	One-to-one user; business name; region/location; verification status
CropCategory	Name, unique slug, active flag
Listing	Seller FK; category FK; name/description; unit code; unit price minor units; currency; status
HarvestBatch	Farmer FK; category/crop; quantity and unit; harvested/created dates; local client ID if needed for sync
StockMovement	Listing FK; movement type; signed quantity delta or documented direction convention; reference; actor; timestamp
Reservation	Listing/order-item FKs; quantity; status; expiry; creation and transition timestamps
CustomerOrder	Buyer FK; seller; status; currency; server-calculated total; timestamps
OrderItem	Order/listing FKs; seller and crop/description snapshots; quantity; unit-price snapshot; line-total snapshot
OrderStatusHistory	Order FK; previous/new status; actor; timestamp; reason/metadata
Dispute	Order FK; opener; reason; status; resolution fields and timestamps
Payment	Order FK; provider/mode; amount minor units; currency; status; idempotency key; timestamps
Refund	Payment FK; amount; reason; status; idempotency key; timestamps
Settlement	Payee; amount; currency; status; reference; simulated settlement metadata
LedgerAccount	Unique code; account type; optional owner; currency; active flag
JournalEntry	Unique reference; source type/id; status; created/posted timestamps; reversal reference if applicable
JournalLine	Journal entry/account FKs; debit and credit minor units; exactly one side positive
SyncOperation	User; client operation UUID; operation type; status; result/error; timestamps
AuditEvent	Actor; action; object type/id; timestamp; safe metadata; append-only policy

Relationships and constraints

- One user may have a farmer profile or buyer profile as appropriate; enforce the intended profile/role relationship in validation and tests.
- Each listing belongs to one seller and one category. A listing's currency and unit must be explicit.
- Each MVP order belongs to one buyer and one seller. Order items preserve listing, seller, crop/description, unit, quantity, unit price, currency and line total as snapshots.
- Do not delete orders, posted payments, journal entries, or audit events as ordinary cleanup. Use protected foreign keys, archival or status transitions.
- Use unique constraints for normalized email, role pairs, slugs, idempotency scope, journal reference, and (user, sync operation ID).

- Use indexes for listing status/category, seller/status, order buyer/date and status/date, reservations listing/status/expiry, payments order/status, ledger account/entry, and audit object/date.
- Use explicit deletion policies. Financial and historical records should generally use PROTECT/RESTRICT rather than cascading deletion.

Money and units

Initial operating currency is NGN. Store money as integer minor units and include a currency code on every monetary aggregate or ledger account. Do not assume all crops use kilograms: represent units explicitly (for example kg, tonne, bag) and prevent invalid comparisons across incompatible units. If unit conversion is supported later, define a controlled conversion table and preserve the unit used on the original order.

6. Business invariants and state machines

These rules are mandatory. They should be implemented in domain services and tested with database-backed tests.

Inventory rules

- Quantities must be positive when submitted as quantities; stock movements must follow one documented sign convention.
- Available quantity = accepted stock on hand minus active, unexpired reservations, using one authoritative calculation path.
- An offline harvest record is pending until server validation and sync acceptance; it must not increase server-authoritative stock before acceptance.
- All stock-affecting operations must be atomic. Lock the relevant listing/inventory row with select_for_update() inside transaction.atomic() when reserving or consuming stock.
- Reject a reservation that exceeds available quantity. Two concurrent requests must not both reserve the same units.
- Reservation expiry must not release stock already consumed, confirmed, or linked to an invalid state transition.
- Do not count both a reservation and a stock deduction for the same event. Document precisely when stock becomes reserved, consumed, returned, or adjusted.

Order lifecycle

State	Meaning / transition rule
draft / pending	Order created and validated by server; stock reservation rules applied
confirmed	Seller/operations confirms according to product rules; reservation remains valid or stock consumption is recorded
fulfilment	Order is being prepared/dispatched/delivered using explicit allowed transitions
completed	Fulfilment completion recorded; financial workflow can be finalised according to simulation rules
cancelled	Cancellation allowed only in defined states; release reservation exactly once
disputed	Dispute is opened and further actions are restricted according to documented policy

The agent must implement an explicit transition map, not accept arbitrary client-provided status strings. Every transition records actor, timestamp, prior state, new state, and reason when applicable. Invalid transitions return a consistent 4xx response.

Order pricing

The server calculates line totals and order totals. Never trust client-submitted totals, unit prices, seller IDs, currency, or discount amounts. Store immutable snapshots so later listing edits do not change an existing order.

Reservation algorithm (illustrative)

```
with transaction.atomic(): listing = Listing.objects.select_for_update().get(pk=listing_id) available
= calculate_available_quantity(listing) if requested_quantity <= 0: raise ValidationError("Quantity
must be positive") if requested_quantity > available: raise InsufficientStock()
create_reservation(...)
```

The agent must adapt this sketch to the final schema and add concurrency tests against PostgreSQL. SQLite tests alone are not sufficient evidence of correct PostgreSQL row-locking behavior.

7. API contract and endpoint plan

All API routes should be versioned under /api/v1/. Define serializers, permissions, pagination, filtering, validation, and consistent error responses. Avoid exposing Django model internals directly.

Area	Endpoint examples	Notes
Authentication	POST /auth/register; POST /auth/login; POST /auth/logout; GET /auth/me	Choose and document session/token strategy; rate-limit sensitive endpoints where appropriate
Catalogue	GET /categories; GET /listings; GET /listings/{id}; POST /listings; PATCH /listings/{id}	Public read policy can be limited; only seller can manage own listing
Harvest	GET/POST /harvest-batches; PATCH /harvest-batches/{id}	Ownership checks; sync status and server validation
Inventory	GET /listings/{id}/availability; GET /stock-movements; POST /reservations	All writes through domain services
Orders	POST /orders; GET /orders; GET /orders/{id}; POST /orders/{id}/transitions; POST /orders/{id}/disputes	Server totals; seller/buyer scope; explicit transitions
Payments	POST /orders/{id}/simulate-payment; GET /payments/{id}; POST /payments/{id}/simulate-refund	Simulation-only; idempotent; no real gateway call
Ledger	GET /finance/ledger-accounts; GET /finance/journal-entries; GET /finance/reports	Restricted to finance/admin roles; never expose secret metadata
Sync	POST /sync/operations; GET /sync/operations/{operation_id}	Idempotent operation receipt; per-user operation scope
Audit	GET /audit-events	Restricted, paginated, filtered; audit access itself may be audited

API response conventions

- Use suitable HTTP status codes and a stable error shape with a machine-readable code, human-readable message, and field errors where relevant.
- Paginate collection endpoints and enforce reasonable page-size limits.
- Apply object-level permissions on every endpoint; hiding buttons in a client is not security.
- Use explicit filters and ordering; do not permit arbitrary database field ordering.
- Document request/response examples and error cases in an OpenAPI schema or equivalent API contract.
- Make create/retry operations idempotent where duplicate submissions could create duplicate orders, payments, refunds, reservations, or ledger postings.

8. Offline-first mobile synchronisation

The .NET MAUI farmer app uses SQLite to preserve locally captured harvest data and pending operations when network access is unavailable. The mobile database is not a second authoritative inventory database.

Local data model

- Store local records with a stable client-generated UUID, local created/updated timestamps, sync status, and last error where relevant.
- Use an outbox table for pending operations; do not rely on a transient in-memory queue.
- Persist before showing a successful local-save confirmation.
- On app restart, resume pending operations and show clear states such as pending, synced, or needs attention.
- Keep server-issued IDs and client-generated IDs distinct where useful; define how a client ID maps to a server record.

Sync protocol

1. Client creates a local record and durable outbox operation with a unique operation UUID.
2. Client sends operation UUID, type, schema/version, payload, and relevant server version if one exists.
3. Server authenticates the user, checks role/ownership, validates payload, and looks up (user_id, operation_id).
4. If the operation was already processed, server returns the stored result rather than applying it again.
5. If new, server applies the operation transactionally, stores the outcome, and returns the canonical server record/result.
6. Client records the result and marks the outbox item synced. Recoverable failures remain queued with bounded backoff; validation conflicts require user attention.

Conflict and retry rules

- Define a conflict policy per entity; never silently overwrite server changes.
- Use server version/updated_at or explicit version numbers for mutable records where concurrent edits matter.
- Retry network failures with exponential backoff and jitter; avoid unbounded rapid retry loops.
- Treat timeouts as ambiguous outcomes and retry using the same idempotency key.
- Do not accept client timestamps as proof of financial or stock event ordering.
- Make server sync responses stable enough that a retry returns the same logical result.

9. Simulated payments and financial ledger

Build the financial structure carefully even though the MVP has no real-money movement. Simulated payment status and ledger entries must remain clearly labelled as simulated throughout the UI, API, reports, and documentation.

Ledger rules

- Use double-entry journal entries: the sum of debits must equal the sum of credits for each posted journal entry and currency.
- Every posted journal has a unique reference and a source reference to the business event that caused it.
- Journal lines require positive integer amounts on exactly one side (debit or credit), never both and never neither.
- Validate whole-entry balance in the controlled posting service; a row-level database check cannot by itself guarantee cross-row balance.
- Posted journal entries and lines are immutable. Correct errors using reversal entries, not edits or deletes.
- Create the journal and update the related payment/business state in one database transaction.
- Use idempotency keys and unique references to prevent duplicate postings on retries or concurrent requests.
- Never keep a database transaction open while waiting on an external network call.

Illustrative simulated payment posting

For a simulated customer payment of ■100,000, an illustrative posting may debit a simulated payment-clearing account by 10,000,000 kobo and credit an order-funds liability account by 10,000,000 kobo. The exact chart of accounts, fee treatment, settlement stages, and liability semantics must be reviewed and documented before implementation. This example is not a statement about real escrow or custody.

Posting service sequence

1. Start transaction.atomic().
2. Lock and validate the payment/order business record.
3. Check idempotency key and existing journal reference.
4. Validate that the current state allows this event.
5. Create journal entry and debit/credit lines.
6. Validate equal totals for each currency and mark the entry posted.
7. Update payment/order state and audit record in the same transaction.
8. Commit; any error must roll back the entire operation.

Refunds and settlements

A refund must not exceed the eligible captured/simulated-paid amount less prior successful refunds. Enforce this under transaction locks and idempotency. Settlement is a separate workflow/state record; do not assume that a payment marked successful means the seller has been settled. For MVP, all such movements are simulated and must never call a live payment API.

10. Security, privacy, reliability and operations

Security requirements

- Never commit secrets, production credentials, private keys, or real customer data. Provide .env.example with placeholder values only.
- Use Django password hashing and supported authentication practices; do not implement custom password cryptography.
- Apply least privilege and role/object-level authorization to all sensitive endpoints.
- Validate input on the server; use Django ORM parameterization and avoid raw SQL unless justified and safely parameterized.
- Protect against insecure direct object references: test that a buyer/farmer cannot access another user's records by changing an ID.
- Configure allowed hosts, CORS, CSRF, secure cookies/TLS, debug mode, and trusted origins appropriately per environment.
- Rate-limit login and other abuse-prone endpoints where appropriate; do not leak whether arbitrary email addresses are registered.
- Do not log passwords, access tokens, payment secrets, or unnecessary personal data.

Reliability and data governance

- Use atomic transactions for multi-record business operations.
- Add health/readiness checks and structured logs with correlation/request IDs.
- Provide backup and restore documentation for PostgreSQL; test restoration before claiming it works.
- Define data retention, account deactivation, archival, and deletion policies. Preserve required financial and audit history.
- Use UTC/timezone-aware server timestamps; display local times at the client boundary.
- Monitor failed syncs, payment simulations, stock conflicts, and background task failures.
- Keep production-like secrets and data out of development fixtures and screenshots.

Deployment readiness

The agent must supply repeatable local setup instructions, database migration commands, test commands, environment-variable documentation, and a deployment checklist. Performance goals should be treated as targets to measure, not as achieved results. Any claim about security, uptime, or throughput must be supported by a test or operational evidence.

11. Implementation roadmap and agent workflow

Milestone	Deliverables	Exit condition
M0 — Inspect and plan	Inspect repository; report current state; list assumptions; create task breakdown and ADRs for major choices	No destructive edits; agreed plan and repo map
M1 — Foundation	Django project, settings, custom User, PostgreSQL config, .env.example, Docker Compose, health endpoint, CI/test setup	Fresh clone starts locally; migrations and tests pass
M2 — Accounts and catalogue	Roles/profiles, crop categories, listings, permissions, pagination/filtering	API tests cover ownership and validation
M3 — Harvest and inventory	Harvest batches, stock movements, reservation service, stock availability	PostgreSQL concurrency tests prove no overselling
M4 — Orders	Order creation, price snapshots, status transition service/history, cancellation and reservation release	Invalid transitions rejected; duplicate requests safe
M5 — Simulated finance	Payment simulation, refund workflow, ledger accounts/journals/lines, audit integration	Balanced journals; idempotent posting; reversal tests
M6 — Sync	SyncOperation/outbox contract, duplicate operation handling, conflict response contract	Repeated sync does not duplicate records or effects
M7 — Dashboard and mobile	Angular buyer/ops workflows; .NET MAUI local capture/outbox and sync UX	End-to-end critical journeys work
M8 — Hardening	Security checks, observability, backup/restore drill, documentation, deployment	Release checklist completed; known gaps documented

Rules for working as an AI coding agent

1. Read this specification and inspect the existing repository before changing files.
2. Summarise the current repository, missing pieces, assumptions, and the next milestone before coding.
3. Work on one milestone or a small vertical slice at a time. Do not create the entire product in a single giant change.
4. Before implementation, list files to add/change and explain important design choices briefly.
5. Write tests alongside business logic. Prefer behavior-focused tests over tests that only mirror implementation details.
6. Run formatting, linting, migrations/checks, and relevant tests; report exact commands and results honestly.
7. Do not claim a command passed if it was not run. Distinguish tested, untested, mocked, and incomplete behavior.
8. Update README and relevant docs whenever setup, endpoints, schemas, or business rules change.
9. Do not silently change product decisions. If a decision materially affects data integrity, legal exposure, payment semantics, or architecture, raise it with options and a recommendation.
10. Keep changes small, reviewable, and reversible. Do not delete existing work without explaining why and obtaining approval where the change is destructive.

12. Testing strategy and acceptance criteria

Minimum test layers

Test layer	Examples
Model / constraint	Unique normalized email; valid statuses; positive quantities; one-sided journal line; required currency/unit
Service / unit	Availability calculation; valid order transition; server price calculation; refund cap; ledger balance
API	Auth/permissions; invalid payloads; pagination; object ownership; stable errors; server-controlled totals
Integration	Order reservation and cancellation; simulated payment + ledger posting atomicity; sync retry
Concurrency (PostgreSQL)	Two buyers competing for limited stock; duplicate payment request; duplicate ledger posting
Mobile sync	Save offline, restart app, retry, handle duplicate response, resolve validation conflict
Security regression	Cross-user access denied; role escalation blocked; secrets not exposed in API/logs

Definition of done for every milestone

- Code follows the agreed architecture and does not duplicate business rules across views/clients.
- Relevant automated tests exist and pass; PostgreSQL-specific correctness tests run against PostgreSQL.
- Database migrations are included and succeed on a clean database.
- Authorization, validation, failure behavior, and idempotency are considered for all write operations.
- Documentation and example environment settings are updated.
- Agent reports what was changed, what commands ran, test results, known limitations, and the recommended next step.

MVP acceptance criteria

- A farmer can create and manage their own listing and harvest records; another user cannot edit them.
- A buyer can browse active listings and create an order using server-derived prices and totals.
- Concurrent reservation requests cannot reserve more than available stock.
- Cancellation or expiry releases a reservation exactly once and never restores stock incorrectly.
- Order status changes follow the allowed transition map and produce history records.
- A simulated payment retry cannot create duplicate payment effects or ledger postings.
- Every posted journal entry balances per currency; posted records cannot be edited/deleted through ordinary APIs.
- A duplicate sync operation returns its existing result and does not apply its effect twice.
- Offline harvest data persists across app restart and visibly distinguishes pending from synced records.
- A clean developer environment can follow the README to install dependencies, configure PostgreSQL, migrate, run tests, and start the API.

13. Explicit non-goals and future expansion

Do not build these as part of the first milestone unless the product owner explicitly changes scope:

- Real-money escrow, custody of funds, or actual payouts to farmers.
- Live Paystack, Flutterwave, bank transfer, or card integration.
- Loans, credit scoring used to approve credit, or automated lending decisions.
- Multi-seller cart/checkout, complex marketplace commission splitting, or cross-border settlement.
- Microservices, Kubernetes, distributed event streaming, or premature multi-region deployment.
- AI-based crop pricing, credit scoring, or yield predictions without a separately approved dataset, evaluation plan, and risk review.
- Claiming that an offline listing is immediately available for online purchase before server sync validates stock.

Expansion-ready design without premature complexity

Use currency codes, country/region fields, explicit units, configurable payment adapters, stable API versioning, and separated business services. Avoid Nigeria-specific assumptions scattered throughout code. Do not implement unused country, tax, or payment features before requirements are defined. Regional expansion will require local legal, currency, payment, language, logistics, and data-protection review.

Key unresolved decisions to raise rather than guess

- Exact order and fulfilment state transitions and who may perform each transition.
- How inventory is represented: whether a listing derives from harvest batches or owns a separate stock balance; choose one consistent source-of-truth strategy.
- Whether a listing can draw stock from multiple harvest batches and how batch traceability works.
- Exact chart of accounts and simulated accounting entries for cancellation, refund, fees, and settlement.
- Authentication method shared by Angular and .NET MAUI clients.
- Required identity verification, privacy/retention rules, and any future regulated payment arrangements.

14. Instructions to the AI coding agent

Copy the following operational instruction into the coding agent's task prompt alongside this PDF:

```
You are the senior engineer implementing AgriFi Trade Network. Treat this specification as the
primary product and architecture contract. Start by inspecting the repository and reporting its
structure, existing implementation, test status, and gaps against this document. Do not overwrite or
regenerate existing work without inspection. Implement the system in small, complete milestones.
Begin with M0 and M1 only unless the project owner explicitly requests a broader scope. Use Python,
Django, Django REST Framework, and PostgreSQL for the backend. Preserve the agreed future clients:
.NET MAUI + SQLite for the offline-first farmer app, and Angular + Tailwind CSS for the
buyer/operations dashboard. Keep the backend as a modular monolith. Put critical business logic in
explicit services. Implement and test ownership permissions, server-calculated prices, safe stock
reservation, valid order transitions, idempotent sync and simulated payment operations, and balanced
immutable ledger postings. Use PostgreSQL-backed tests for row-locking/concurrency claims. Never use
floating-point for money. The first release must use simulated payments only. Do not call a live
payment provider, move real funds, describe the system as legally providing escrow, implement
lending, or claim compliance without explicit approval and the appropriate external review. For each
milestone: (1) state the goal and assumptions; (2) list planned files and migrations; (3) implement
the smallest vertical slice; (4) add tests; (5) run and report exact checks; (6) update
documentation; (7) list remaining risks and the next recommended milestone. Be candid about anything
not run, not tested, mocked, or incomplete. Never fabricate successful tests, production readiness,
security guarantees, or performance results. If an important requirement is ambiguous, explain the
competing options, recommend one, and ask for a decision when choosing incorrectly could compromise
data integrity, financial correctness, privacy, legal exposure, or irreversible architecture.
Otherwise choose a conventional low-complexity default and document it.
```

First task for the agent

> Inspect the repository and produce a short implementation plan for M1. Then establish the Django project, custom user model, PostgreSQL configuration, environment-variable template, health check, initial migration, and a minimal automated test. Do not proceed to orders, payments, or the mobile/web clients until the foundation is reproducible.

Final instruction

This specification is intentionally conservative about money, inventory, and offline synchronization. The agent should optimise for correctness, traceability, and a working, testable foundation—not for the largest amount of generated code.

End of AgriFi Trade Network AI Agent Build Specification · Version 1.0